# Capitolo 1

# Fondamenti di ingegneria del software e qualità

## 1.1 Ingegneria del software e prodotto software

L'ingegneria del software usa metodi, strumenti e pratiche organizzate per produrre il software, gestirlo in esercizio e farlo evolvere. Il suo obiettivo non è soltanto ottenere un programma che produca un risultato oggi, ma un prodotto che continui a soddisfare bisogni espliciti anche quando cambiano requisiti, persone, piattaforme e dati.

Il software è infatti immateriale e modificabile, ma questa flessibilità non rende gratuiti i cambiamenti: una modifica può introdurre difetti, contraddire una decisione precedente o rendere il codice meno comprensibile.

#### Prodotto software e artefatti

Un **prodotto software** comprende il programma eseguibile, il codice sorgente, i dati e gli artefatti necessari per comprenderlo, usarlo, verificarlo e modificarlo.

Un **artefatto** è il risultato di un'attività: sono artefatti una specifica dei requisiti, un diagramma UML, il codice, una configurazione di build, un piano di test e una guida utente.

La qualità dipende anche dalla coerenza fra questi artefatti: un test non può verificare correttamente un requisito ambiguo, e un'implementazione non è facilmente manutenibile se il suo modello è sconosciuto o non è più aggiornato.

Nel ciclo di vita si riconoscono attività ricorrenti, non necessariamente svolte una sola volta né in un unico ordine:

1. comprendere bisogni, vincoli e criteri di accettazione;
2. definire e convalidare requisiti verificabili;
3. progettare architettura, responsabilità, interfacce, dati e algoritmi;
4. implementare, integrare e controllare il prodotto;
5. consegnare, usare, correggere ed evolvere il software.

Un processo di sviluppo decide come coordinare queste attività, chi produce gli artefatti e quando si raccolgono feedback. Requisiti, progetto, codice e test restano collegati durante

l'intero ciclo di vita: cambiare un requisito può richiedere di modificare sia il programma sia i controlli usati per verificarlo.

## 1.2 Correttezza, requisiti e attributi di qualità

Un **requisito** descrive una proprietà o un servizio richiesto al sistema. Distinguiamo i requisiti funzionali, che indicano ciò che il sistema deve fare, dai requisiti non funzionali, che impongono vincoli o livelli di qualità. Prestazioni, affidabilità, sicurezza e manutenibilità appartengono a questo secondo ambito.

Per verificare un requisito occorre sapere in quali condizioni osservarlo e che cosa misurare. La frase "il sistema deve essere veloce" non precisa contesto, carico e misura. Un requisito verificabile può invece indicare il tempo massimo di risposta per una certa operazione e per un certo numero di utenti.

La **correttezza** indica se il comportamento del sistema rispetta una specifica: per gli input ammessi, il sistema deve produrre i risultati e gli effetti previsti. La correttezza è necessaria, ma da sola non basta a definire la qualità. Un programma può calcolare correttamente un risultato e tuttavia essere difficile da usare, troppo lento, vulnerabile o quasi impossibile da modificare.

La **qualità del software** è quindi l'insieme delle proprietà che rendono il prodotto adatto al suo uso e alla sua evoluzione. La qualità va valutata rispetto ai requisiti, al contesto e ai compromessi del progetto.

Tabella 1.1: Attributi di qualità e domande di controllo.

| Attributo | Domanda di controllo |
|---|---|
| Correttezza | Il comportamento osservato soddisfa la specifica? |
| Affidabilità | Il servizio resta disponibile e produce risultati corretti nelle condizioni previste? |
| Efficienza | Tempi, memoria e altre risorse rispettano i vincoli dichiarati? |
| Usabilità | Gli utenti riescono a comprendere e usare il sistema per i propri obiettivi? |
| Manutenibilità | È possibile comprendere, correggere, testare e modificare il sistema con costo controllato? |
| Portabilità | Il sistema può essere trasferito o adattato all'ambiente richiesto? |

Gli attributi non sono indipendenti: aggiungere controlli può aumentare affidabilità e costo di esecuzione; una generalizzazione può facilitare estensioni future ma rendere il progetto iniziale più complesso. Perciò non esiste una qualità assoluta separata dallo scopo del sistema. I requisiti non funzionali chiariscono quali compromessi devono essere accettati.

### 1.2.1 Dependability e compromessi fra manutenibilità ed efficienza

La **dependability** è la capacità di offrire un servizio su cui si possa riporre una fiducia giustificata. Per capire da che cosa dipende questa fiducia, occorre distinguere tre aspetti.

La *security* riguarda la protezione dei dati e degli accessi. La *safety* riguarda l'assenza di pericoli inaccettabili per persone e cose. La *reliability* esprime la probabilità di funzionare senza guasti per un intervallo nelle condizioni previste.

Queste proprietà rispondono a problemi diversi: un sistema può proteggere bene gli accessi ma avere guasti frequenti, oppure funzionare regolarmente senza essere sicuro rispetto agli attacchi. I tre termini non sono quindi intercambiabili.

La manutenibilità ha un ruolo centrale nel ciclo di vita: un prodotto di successo può sopravvivere all'hardware iniziale e ricevere sempre più richieste di estensione.

L'efficienza riguarda soprattutto tempo di CPU, memoria e altre risorse. Un'ottimizzazione che rende più difficile leggere e modificare il codice deve essere motivata da un vincolo concreto; nell'High Performance Computing (HPC), per esempio, l'importanza delle prestazioni può giustificare compromessi differenti.

## 1.3 Responsabilità, coesione e accoppiamento

La struttura interna influenza direttamente la manutenibilità. Una **responsabilità** è un compito coerente assegnato a un elemento del sistema. Nella progettazione a oggetti una *classe* descrive i dati e le operazioni dei propri oggetti: assegnare responsabilità alla classe significa decidere quali informazioni conserva e quali compiti svolge.

Il Principio di Singola Responsabilità (SRP) suggerisce che un elemento abbia un motivo principale per cambiare: se una classe calcola importi, gestisce la persistenza e produce schermate, modifiche dovute a ragioni indipendenti tendono a concentrarsi nella stessa classe.

La **coesione** esprime quanto dati e operazioni di un componente siano rivolti allo stesso scopo; una coesione elevata rende più semplice attribuire e comprendere la responsabilità.

L'**accoppiamento** esprime quanto un componente dipenda da altri componenti. La coesione riguarda dunque i legami interni a un componente; l'accoppiamento riguarda le dipendenze fra componenti.

Gli oggetti devono collaborare, perciò la presenza di accoppiamento non è di per sé un errore. Il problema sono le dipendenze inutili: quando cambia un componente, queste possono costringere a modificarne altri.

L'**incapsulamento** limita la conoscenza della rappresentazione interna, mentre un'*interfaccia* espone le operazioni con cui collaborare. Il contratto di tali operazioni stabilisce che cosa un oggetto offre e a quali condizioni: chi lo usa può dipendere dal servizio senza conoscere ogni dettaglio della sua realizzazione.

Lo *spaghetti code* è una descrizione informale di codice con flusso e dipendenze difficili da seguire, spesso ricco di duplicazioni, condizioni annidate e responsabilità mescolate. Non è una metrica né una diagnosi: segnala che occorre esaminare struttura, test e requisiti.

Il *debito tecnico* indica il costo futuro generato da una soluzione rapida ma poco curata. Non coincide con ogni scelta pragmatica: diventa debito quando la scorciatoia non viene resa esplicita e ostacola sistematicamente le modifiche successive.

### 1.3.1 KISS, DRY, YAGNI e Law of Demeter

**KISS: Keep It Simple, Stupid.** Scegliere una soluzione semplice da comprendere e gestire. La complessità non è una prova di qualità; un'astrazione è utile quando semplifica un problema reale.

**DRY: Don't Repeat Yourself.** Evitare di duplicare la stessa conoscenza. Se una regola è copiata in più metodi, una correzione deve essere ripetuta ovunque e rischia di lasciare versioni incoerenti. Estrarre un metodo può localizzarla.

**YAGNI: You Ain't Gonna Need It.** Implementare ciò che serve ai requisiti esistenti. Preparare varianti speculative aumenta codice, test e dipendenze senza un beneficio già richiesto.

**LoD: Law of Demeter.** Limitare la conoscenza dei dettagli degli oggetti lontani: comunicare soprattutto con i collaboratori diretti. Una catena che attraversa molti oggetti per raggiungere e modificare un dato espone la struttura interna e favorisce cambiamenti a cascata.

#### Accoppiamento: una soglia orientativa

**Precisazione**

Un ordine di grandezza di tre–cinque dipendenze può essere usato come euristica, cioè come soglia orientativa per riesaminare l'accoppiamento, ma la LoD non stabilisce un limite numerico universale. Occorre distinguere una classe che conosce troppe altre classi da un'astrazione stabile usata da molti client, cioè da molti utilizzatori. Anche incroci e cicli in un diagramma sono segnali da interpretare in base alle responsabilità effettive.

### 1.3.2 Tell, Don't Ask e separazione command–query

*Tell, Don't Ask* suggerisce di assegnare all'oggetto che possiede i dati l'operazione coerente con quei dati, evitando che il client legga tutto lo stato e ricostruisca fuori dall'oggetto la logica che usa quei dati. La *Command–Query Separation* (CQS) distingue invece due tipi di operazione:

- una *query* restituisce informazioni e non modifica lo stato osservabile;
- un *command* modifica lo stato e, nella forma didattica rigorosa, non restituisce un risultato applicativo.

Per esempio, `getSomma()` osserva un risultato e `inserisci(...)` aggiorna un catalogo. La separazione rende espliciti gli effetti e facilita test e riuso.

Il pattern Command rappresenta una richiesta mediante un oggetto. Aggiunge quindi un'idea distinta dalla CQS: la CQS separa gli effetti dei metodi, mentre il pattern dà alla richiesta una rappresentazione su cui altri oggetti possono operare.

**Precisazione**

I sintomi di cattiva programmazione comprendono metodi lunghi che usano stato globale, classi con nomi di procedure, poche collaborazioni significative e duplicazioni. Un modello a oggetti esprime responsabilità e relazioni, anziché usare le classi come semplici contenitori di funzioni. Ciò non obbliga ogni classe a essere mutabile: oggetti immutabili, servizi senza

stato e funzioni pure possono essere appropriati. Si valutano responsabilità e contratti, non il semplice fatto che siano presenti campi modificabili.

## 1.4 Verifica, validazione e cambiamento controllato

La **verifica** controlla se gli artefatti rispettano le loro specifiche e le decisioni di progetto. La **validazione** controlla invece se il prodotto risponde al bisogno dell'utente. Il criterio cambia: nel primo caso il riferimento è ciò che è stato specificato, nel secondo è il bisogno da soddisfare.

Le due attività si sostengono a vicenda. Requisiti chiari permettono di costruire test significativi; i risultati dei test e il feedback degli utenti possono, a loro volta, rivelare che i requisiti sono incompleti o non realistici.

#### Test e verifiche statiche

I test eseguono il software e confrontano un risultato osservato con un *oracle*, cioè un risultato o una proprietà attesa. Le verifiche statiche, come revisione, compilazione e analisi del codice, controllano il software senza eseguirlo. In generale, nessuna singola tecnica dimostra l'assenza di tutti i difetti: la fiducia nel software deriva dall'uso coordinato di specifiche, revisione, test a livelli diversi e controllo delle modifiche.

Il miglioramento strutturale deve essere disciplinato. Se un comportamento è corretto ma il codice è difficile da modificare, il *refactoring* interviene sulla struttura senza alterare il comportamento osservabile.

I *test di regressione* rieseguono controlli già superati per individuare effetti indesiderati delle modifiche. Se cambia il requisito, invece, occorre prima aggiornare la specifica e i criteri di verifica: non basta conservare il comportamento precedente.

### 1.4.1 Strumenti di IA e responsabilità del progettista

L'intelligenza artificiale (IA) generativa può produrre codice, ma il risultato richiede progettazione e verifica: può contenere difetti e vulnerabilità, e può richiedere di individuare e correggere i problemi con il debugging.

Riutilizzare soluzioni note e progettare una soluzione per un problema nuovo richiedono valutazioni diverse. Comprendere requisiti, architettura e contratti aiuta a formulare richieste precise e a valutare l'implementazione ottenuta. Le capacità degli strumenti possono evolvere; la disponibilità di codice generato non dimostra, da sola, la correttezza del sistema complessivo.

## 1.5 Dai bisogni a codice e test: procedimento operativo

Per affrontare una funzionalità in modo coerente con i principi del capitolo si può seguire questa sequenza:

1. formulare il bisogno e trasformarlo in requisiti funzionali e non funzionali verificabili;
2. identificare gli artefatti coinvolti e le responsabilità principali;

3. progettare contratti e dipendenze, mantenendo coese le responsabilità e limitando l'accoppiamento non necessario;
4. implementare in passi piccoli e controllabili;
5. definire casi di test con input, stato iniziale, oracle e risultato atteso;
6. integrare e raccogliere feedback; se il requisito cambia, aggiornare in modo coerente specifica, modello, codice e test;
7. misurare e riesaminare le parti che mostrano complessità o dipendenze problematiche, quindi applicare manutenzione o refactoring quando necessario.

In un processo iterativo questa sequenza può essere ripetuta per ogni incremento, mantenendo esplicito il rapporto fra bisogno, decisioni di progetto e verifica.